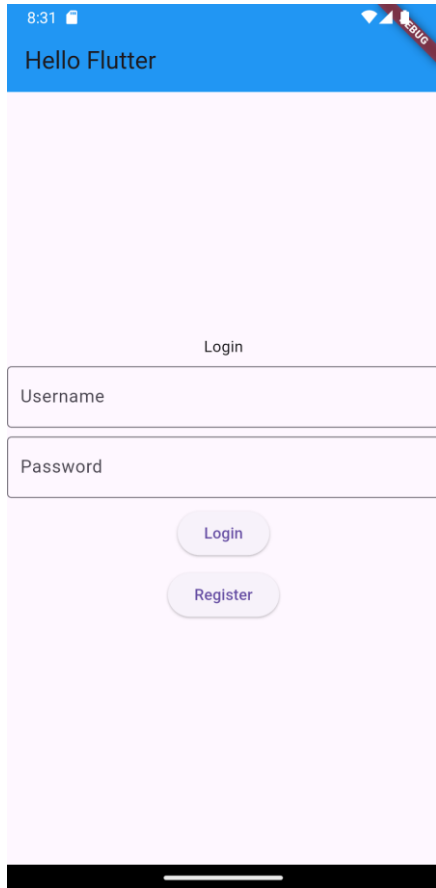
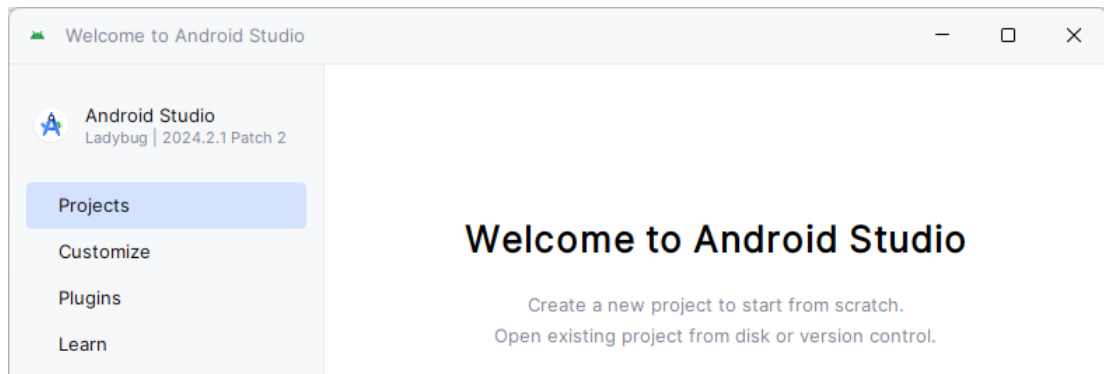


Practical 1: Setup Flutter Development Environment

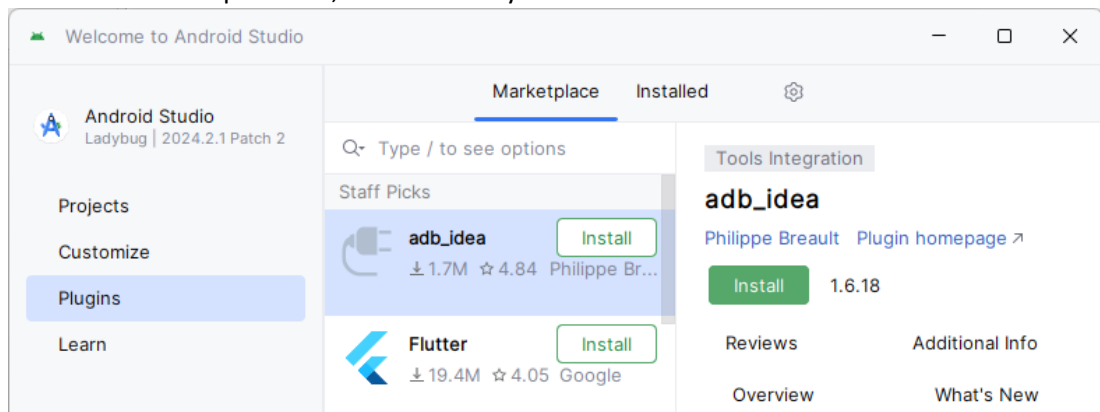
Flutter is Google's UI toolkit for building applications for mobile, web, and desktop from a single codebase. In this practical, you will install the necessary files and build the following Flutter application:



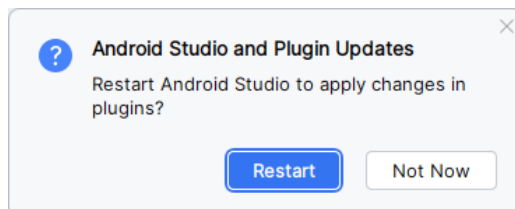
1. Download the following installation files:
 - a. Flutter SDK (<https://docs.flutter.dev/get-started/install>)
 - b. Android Studio (<https://developer.android.com/studio>)
 - c. Git [Optional] (<https://git-scm.com/downloads>)
 - d. Node.js [Optional] (<https://nodejs.org/en/download/current>)
2. Extra the Flutter SDK to a local drive. E.g. C:\flutter\
3. Install Android Studio.
4. Launch Android Studio and install the following components:
 - a. Android SDK Platform
 - b. Android SDK Command-line Tools
 - c. Android SDK Build-Tools
 - d. Android SDK Platform-Tools
 - e. Android Emulator
5. Upon completion of Android Studio installation, on the welcome screen, click **Plugins**.



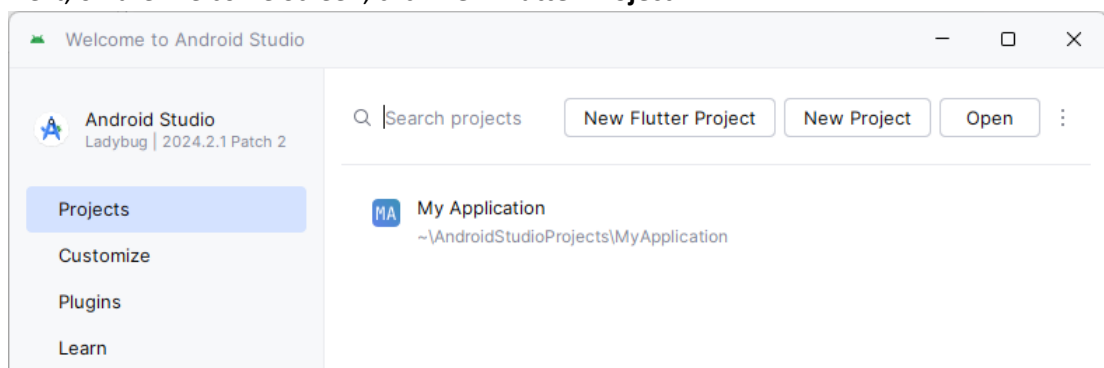
6. Under the Marketplace tab, search the keyword **flutter**.



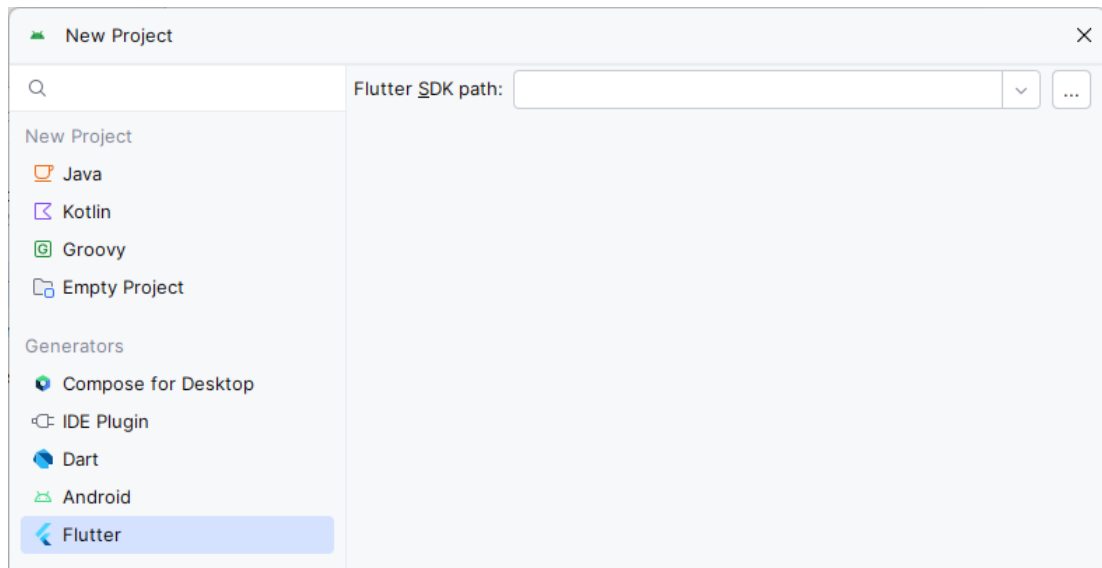
7. Click the Install button next to Flutter. Once the installation is completed, press the Restart IDE and “Restart”.



8. Next, on the Welcome Screen, click **New Flutter Project**.



9. Click **Flutter** on the left panel. Enter the **Flutter SDK path** (E.g. C:\flutter\) and click **Next** button.



10. Next, you will be prompted to enter a project name and select other settings. Choosing a good flutter project name:

Field	Description
Project name	Use lowercase letters and underscores. E.g. my_todo_app.
Project location	A dedicated folder for your Flutter project.
Description	A readme file.
Project type	Determine the kind of application you are building.
Organization	Use your company's domain as a prefix for the project name. Example: com.yourcompany.myapp This helps to organize projects and avoid naming conflicts.
Android language	Kotlin and Java
Platforms	Select the platforms you want to support (Android, iOS, Web, Desktop).
Module name	Same as the project name.

Type	Description
Application	Creates a standalone Flutter application that can be deployed to various platforms like Android, iOS, Web, Windows, and macOS.
Plugin	To provide access to platform-specific APIs (e.g., camera, sensors, Bluetooth) from within your Flutter code.
Package	To create reusable code modules that can be shared across multiple Flutter projects or published to pub.dev (Flutter's package repository).

Module	Creates a reusable Flutter module that can be integrated as a dependency into other Flutter projects.
Skeleton	A simplified project with minimal boilerplate code.
FFI Plugin	To interact with native libraries (C, C++, etc.) using the Foreign Function Interface (FFI) in Dart.
Empty Project	Creates a completely empty project with no default files or structure.

11. For this practical we select an **Empty Project** type.

New Project

Project name: hello_flutter

Project location: ~\AndroidStudioProjects\hello_flutter

Description: A new Flutter project.

Project type: Empty Project

Organization: com.example

Android language: ☐ Java ☒ Kotlin

Platforms: ☒ Android ☒ iOS ☒ Linux ☒ MacOS ☒ Web ☒ Windows

When created, the new project will run on the selected platforms (others can be added later).

☐ Create project offline

More Settings

Module name: hello_flutter

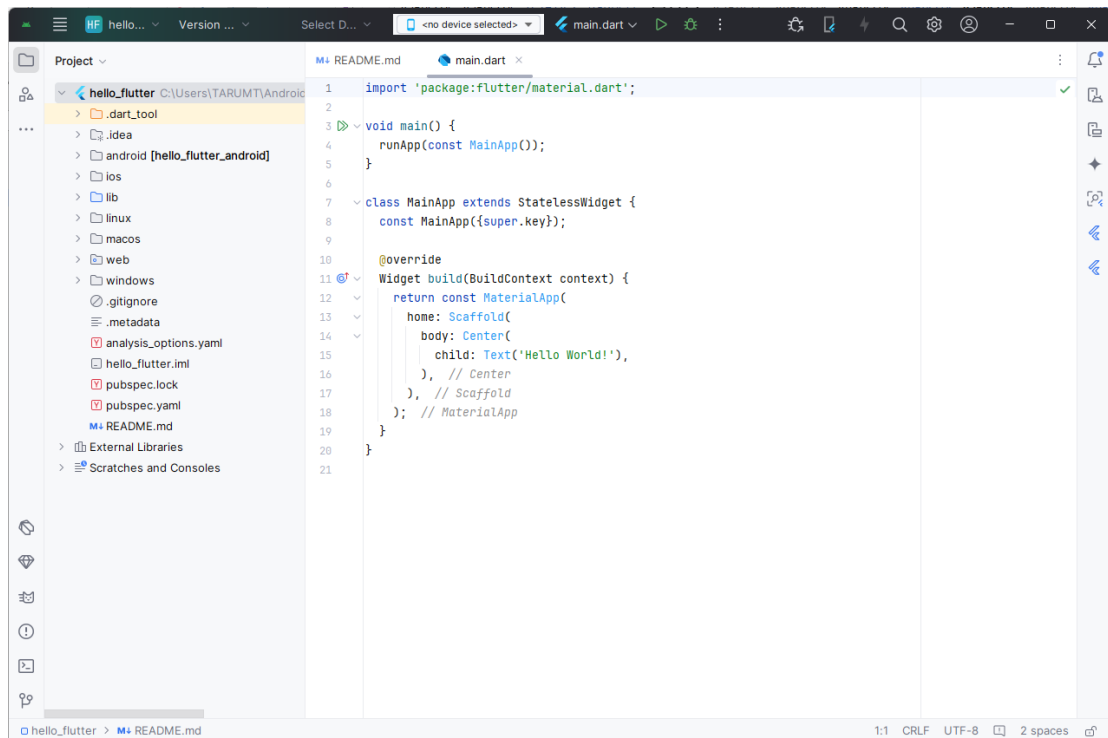
Content root: C:\Users\TARUMT\AndroidStudioProjects\hello_flutter

Module file location: C:\Users\TARUMT\AndroidStudioProjects\hello_flutter

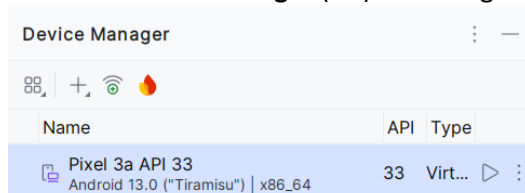
Project format: .idea (directory-based)

Previous Create Cancel

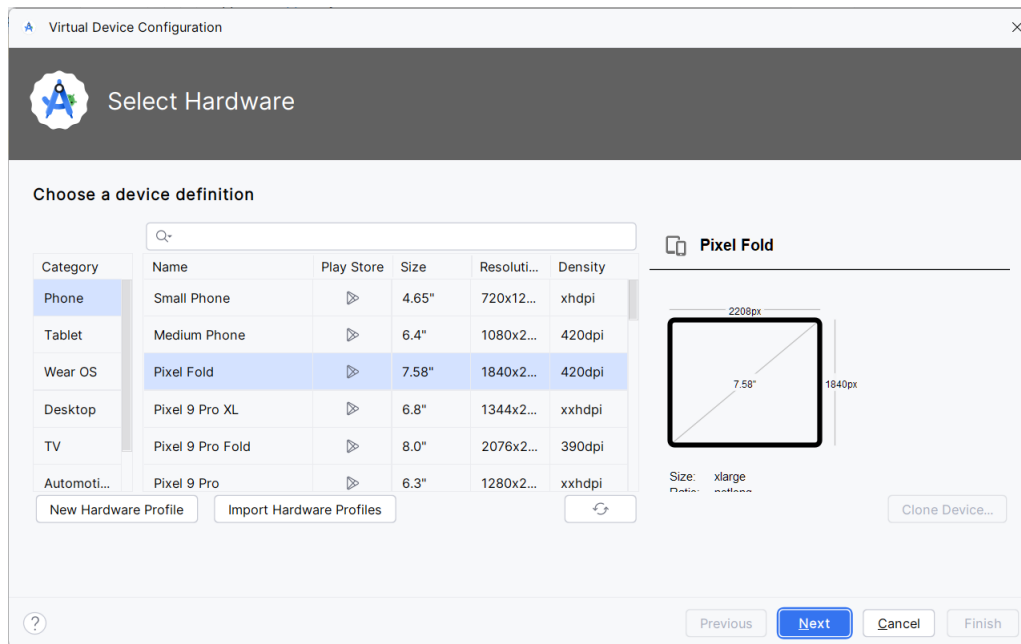
12. Once the project is ready you will see the screen as follows: -



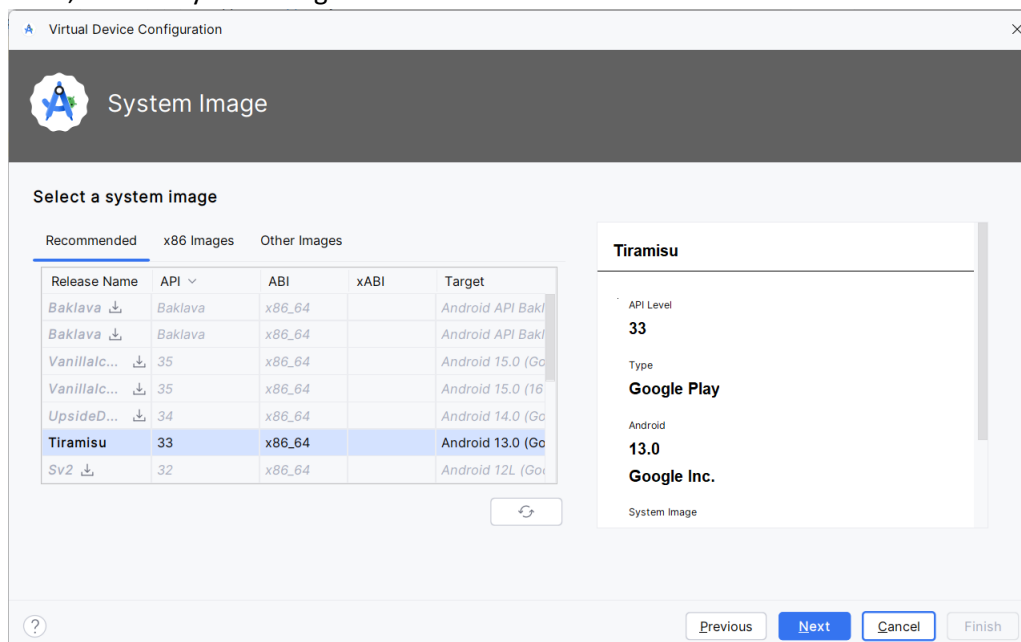
13. Click the **Device Manager** (📱) on the right panel and press the **Run** (▶) button.



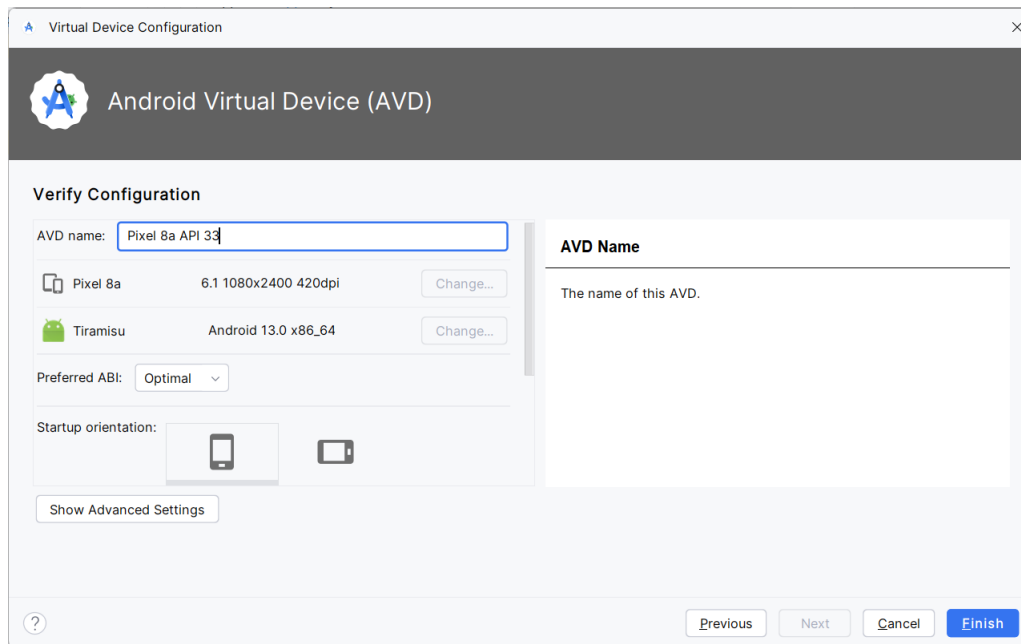
However, If you do not see any device in the **Device Manager**, you may click the **Add a new device** (+) button and click **Create Virtual Device**. Select a device definition from the list and click the **Next** button.



Then, select a system image and click the Next button.



Enter a name in the **AVD** (Android Virtual Device) and click the **Finish** button.

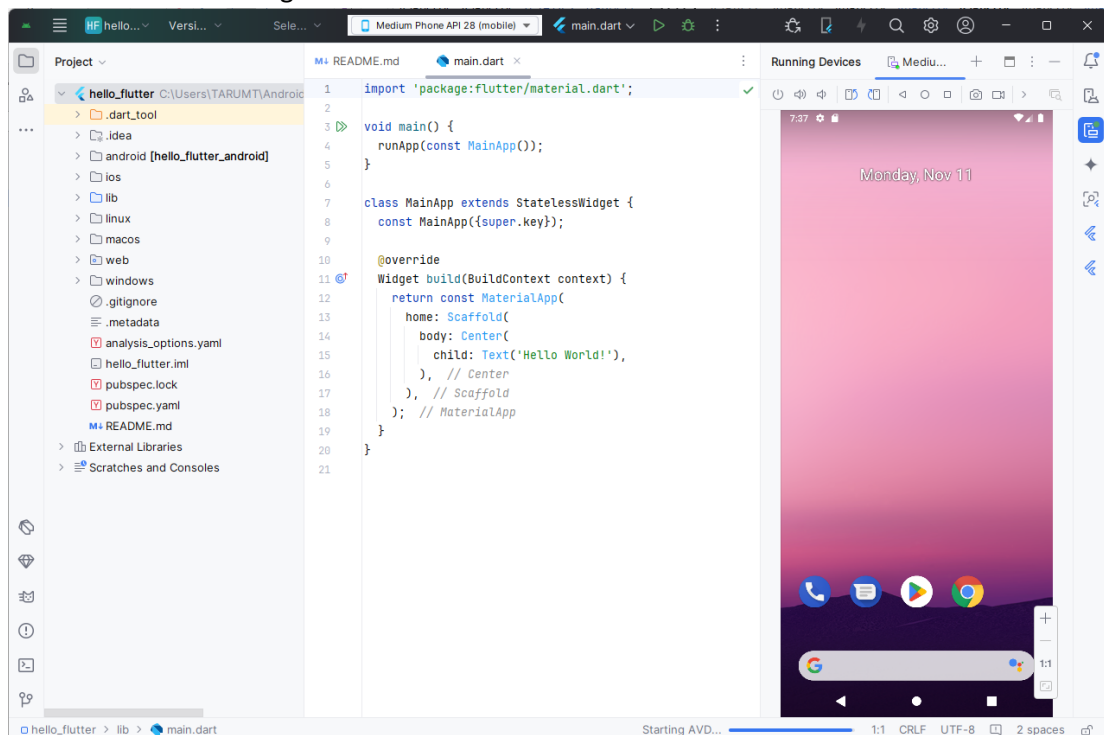


The newly created AVD should appear in the Device Manager.

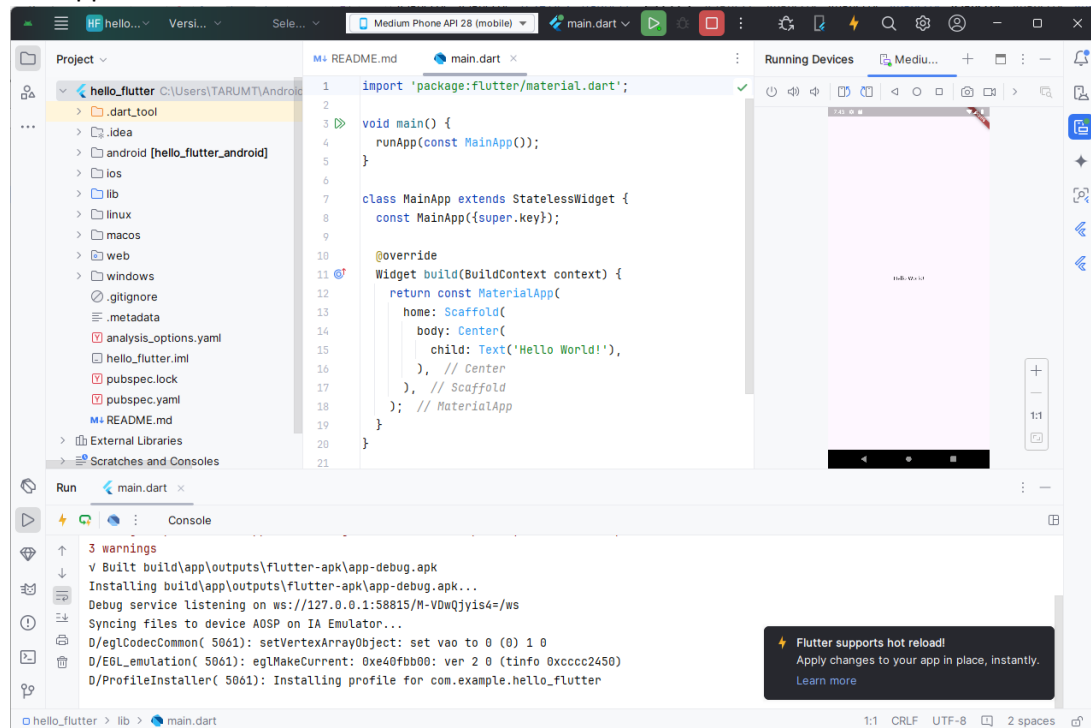
14. On the top of the screen, press the **Run** (▶) button. Running the app for the first time will take a long time depending on the quality of the network.



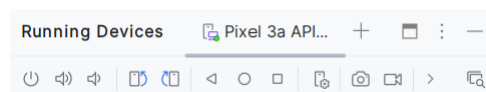
15. You will see the running device as follows: -



16. The app run as follows: -

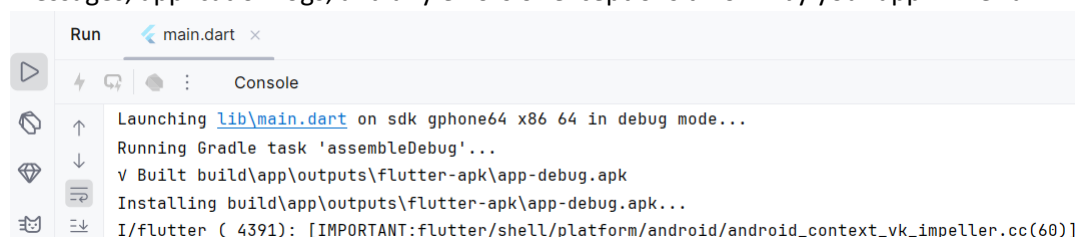


In the **Running Devices** tab () , examine the top panel that has a list of common functions associated with a mobile device such as On/Off button, volume up/down, etc.

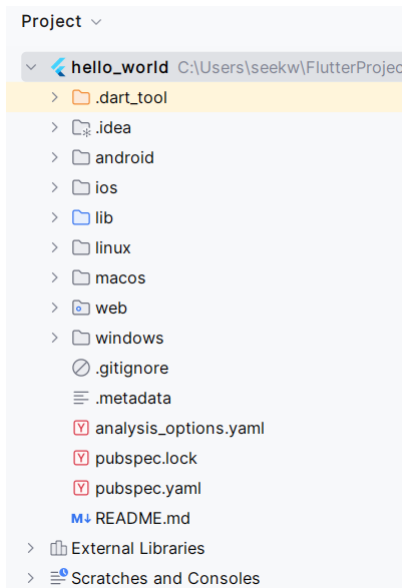


17. At the bottom left of the screen, click the **Run** tab () , which shows the console messages. When you build your app (compile it into an APK or bundle), the console shows the progress of the build process, any warnings or errors encountered, and the final build results.

This panel also shows the logcat output. This is crucial for debugging. It displays system messages, application logs, and any errors or exceptions thrown by your app while running.



18. The project File Explorer is on the left. It lists all the platforms supported by your project.



19. Expand the **lib** folder. This folder contains Dart program files for your project. Open and examine the **main.dart** file and enter the following comments so that you understand the purposes of these commands:

```
//Importing the necessary packages
import 'package:flutter/material.dart';

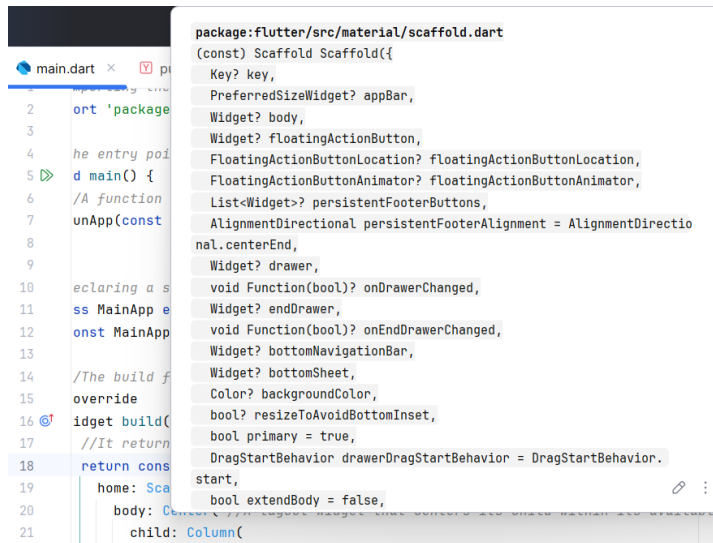
//The entry point of the Flutter application.
void main() {
  //A function that takes a Widget as an argument and renders it on the screen.
  runApp(const MainApp());
}

//Declaring a stateless widget class that does not change its state over time
class MainApp extends StatelessWidget {
  const MainApp({super.key}); //Constructor of this widget

  //The build function is responsible for building the widget's UI
  @override
  Widget build(BuildContext context) {
    //It returns a MaterialApp widget
    return const MaterialApp(
      //A basic layout structure consisting an app bar, a body and drawers
      home: Scaffold(
        //A layout widget that centres its child within its available space
        body: Center(
          child: Text('Hello World!'), //A text widget
        ),
      ),
    );
  }
}
```

Next, let us edit the program code further to change the UI.

20. Mouse over the **Scaffold** widget to observe its attributes. You may scroll down to see all attributes and a short description about the widget.



21. Within the **Scaffold** widget, enter the text **app**, the editor will find and show a matching attribute, press the **Tab** key to accept the suggested attribute.



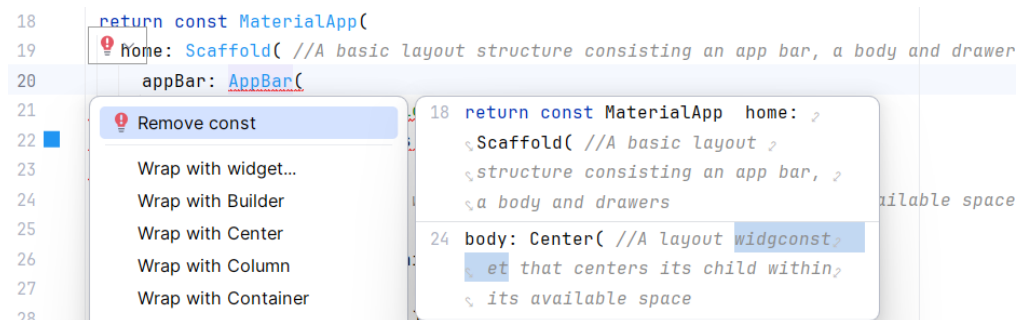
22. Enter the following code (ignore the error at the moment):

```

20 appBar: AppBar(
21   title: const Text('Hello Flutter'),
22   backgroundColor: Colors.blue,
23 ), // AppBar

```

23. If there is an error in your code, the editor might have solutions to fix it. Click any part of the error code and then click the **red light bulb** (💡) to see the possible fixes. Select **Remove const** as a fix.



As suggested by the editor, the command **const** (it means constant) shall be removed from the **MaterialApp** widget after inserting the **AppBar**. The error arises because the **AppBar** widget within the **Scaffold** is not a constant constructor.

After applying the fix your code shall look like this:

```
return MaterialApp(
  home: Scaffold( //A basic layout structure
    appBar: AppBar(
      title: const Text('Hello Flutter'),
      backgroundColor: Colors.blue,
    ), // AppBar
```

24. Save the program code (Ctrl+S) and observe the app bar. Flutter supports hot reload; therefore, you should see the UI changes without the need to relaunch your app.



25. At the moment, the **Center** widget can only hold one child widget. We need another widget that can hold multiple children. Mouse click on the **Text** widget, then click on the **yellow light bulb** (💡) on the right. You will see a drop-down list, select **Wrap with Column**. You can preview each selection on the right of each option.

```
19 home: Scaffold( //A basic layout structure consisting an app bar, a bo
20   body: Center( //A layout widget that centers its child within its av
21     child: Text('Hello World!'), //A text widget
22
23
24
25
26
27
```

Wrap with widget...
 Wrap with Builder
 Wrap with Center
 Wrap with Column
 Wrap with Container
 Wrap with Padding
 Wrap with Row
 Wrap with SizedBox
 Wrap with StreamBuilder

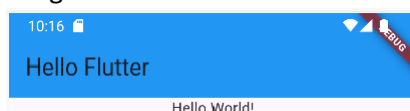
```
21 child: Column(
22   children: [
23     Text('Hello World!'),
24   ],
25 ), //A text widget
```

26. The result will be as follows: -

```
body: Center( //A layout widget that centers its
  child: Column(
    children: [
      Text('Hello World!'),
    ],
  ), //A text widget // Column
), // Center
```

Column is a widget that can contain multiple widgets within the array attribute named **children**.

27. Save your work and view the UI. Now, the **Text** widget is the only child under the **Column** widget.



28. Modify the **Column** widget so that all the children widgets are aligned to the centre of the screen as follows: -

```
child: Column(
  //A text widget
  mainAxisAlignment: MainAxisAlignment.center,
  children: [
    Text('Hello World!'),
  ],
), //A text widget
```

Save your work and observe the result.

29. Modify the existing Text widget. Also insert two TextField widgets to allow users to enter text.

```
Text('Login'),
TextField(
  keyboardType: TextInputType.name,
  decoration: const InputDecoration(
    labelText: 'Username',
    border: OutlineInputBorder(),
  ),
),
TextField(
  keyboardType: TextInputType.text,
  obscureText: true,
  decoration: const InputDecoration(
    labelText: 'Password',
    border: OutlineInputBorder(),
  ),
),
```

30. Insert another two ElevatedButton widgets:

```
ElevatedButton(
  onPressed: () {}, //Empty event handler
  child: Text('Login'),
),
ElevatedButton(
  onPressed: () {}, //Empty event handler
  child: Text('Register'),
),
```

Congratulations! You have completed your first Flutter app!

Remarks:

If you are unable to deploy a Flutter app in Android, do check the Gradle compatibility matrix at <https://docs.gradle.org/current/userguide/compatibility.html>

For Kotlin version 1.8, the minimum Gradle version is 8.0 with JVM between 8 and 23. JVM 24 and later versions are not yet supported.



TODO: Visit <https://dart.dev/language> and learn the basics of Dart language!